

AI Creative Studio

Technikai architektúra — egyszerű leírás a fejlesztőcsapatnak

1. Mit építünk

Egy webes alkalmazást, ami Instagram és Facebook posztokat készít automatikusan, AI segítségével, de úgy, hogy az eredmény mindig hűen tükrözze a felhasználó márkáját.

A felhasználó beírja a weboldala címét. Az alkalmazás 90 másodperc alatt kinyeri a márkája lényegét (színek, logó, betűtípus, hangnem, képi világ). Innentől a felhasználó csak egy mondatos briefet ír, és négy-tíz kész, márkahű poszt érkezik vissza. Egy kattintással jóváhagyja, ütemezi vagy posztolja.

A kulcs gondolat: az AI azt csinálja, amiben jó (szöveg, koncepció, képi hangulat), a többit (logó pontos elhelyezése, pontos színek, betűtípus) deterministikusan sablonok kezelik. Így az eredmény soha nem lesz "AI-szlop" érzetű.

2. A rendszer öt rétege

Az alkalmazás öt különálló rétegből áll. Mindegyik egy-egy konkrét feladatot lát el. A rétegek üzeneteken keresztül beszélnek egymással, és önállóan skálázhatók.

Frontend réteg — amit a felhasználó lát

A webes felület, ahol a felhasználó megnyitja az alkalmazást, létrehozza a brand kitjét, briefeket ír, és átnézi a kész posztokat. Itt zajlik a regisztráció, a csomagváltás, a csapattagok meghívása és a kreatívok jóváhagyása. Ez egy modern webalkalmazás, amit a böngészőből ér el a felhasználó — nincs külön letöltendő szoftver.

Adatréteg — ahol minden tárolódik

Itt él a felhasználói fiókok, szervezetek, brandek, brand kitek, briefek, kreatívok és minden egyéb adat. Központi adatbázisként szolgál a teljes rendszerhez. A márkás képek, logók, és a kész renderelt posztok külön fájl tárolóban tárolódnak (mert nagy méretűek), de a metaadataik az adatbázisban élnek.

AI réteg — az ötletek és a szövegek

Két dolgot csinál. Egyik feladata: amikor egy új felhasználó beírja a weboldala címét, ez a réteg "olvassa el" a weboldalt, és találja ki, mi a márká színpalettája, hangneme, képi világa. A másik feladata: amikor egy brief érkezik, ez találja ki, mit írjon a poszt, melyik sablon illik hozzá, és milyen képet társítsunk hozzá. Ehhez nagy nyelvi modelleket használ (Claude), és képgeneráló modelleket (Flux).

Render réteg — a kész poszt elkészítése

Ez a réteg veszi az AI által kitalált szövegeket, képeket, és a felhasználó brand kitjét, és mindezt összeállítja egy konkrét képpé. Úgy működik, mint egy automatizált grafikus: van egy sablon (pl. quote poszt), van benne szöveghely, képhely, logóhely — a render réteg behelyettesíti ezeket és lement egy 1080×1350 pixel méretű PNG-t. A logó, a színek, a betűtípus innen kerülnek pontosan a helyükre, nem az AI-tól. Ez biztosítja a márkahűséget.

Háttér-feldolgozó réteg — a hosszabb feladatok futtatása

Az olyan feladatok, amik tovább tartanak pár másodpercnél (egy weboldal feldolgozása, négy poszt egyidejű generálása, egy poszt ütemezett kiküldése), nem futhatnak közvetlenül a felhasználó kattintására. Ez a réteg veszi sorra a feladatokat, párhuzamosan dolgozza fel őket, és ha hibázik, automatikusan újrapróbálja. Olyan, mint egy karmester: koordinálja, hogy a többi réteg mikor mit csináljon.

Áttekintő táblázat

Réteg	Mit csinál	Mire figyel
Frontend	A felhasználó interakciói	Sebesség, kényelem, mobil-barát
Adatréteg	Minden adat tárolása	Tenant-izoláció (egy ügyfél ne lássa a másikat)
AI réteg	Szöveg, ötlet, kép generálás	Költséghatékonyság, válaszidő
Render réteg	Sablon + adat = kész poszt	Pixel-pontos eredmény, márkahűség
Háttér-feldolgozó	Hosszú feladatok futtatása	Megbízhatóság, retry-logika

3. Hogyan zajlik egy felhasználói folyamat

Három fő folyamat van a rendszerben. Mindegyiket lépésről lépésre végigvesszük.

Folyamat 1 — Új felhasználó belép, és 5 percen belül posztot készít

- Regisztráció.** Email vagy Google fiókkal — 30 másodperc.
- Forrás megadása.** A felhasználó beírja a weboldalát, vagy összeköti az Instagram fiókját.
- Automatikus brand-kinyerés (60–90 mp).** A háttér-feldolgozó réteg elindít egy feladatot: letölti a weboldalt, kinyeri belőle a logót (favicon, hero kép, header logó), a fő színeket (klaszterezéssel a képekről és CSS-ből), a betűtípusokat (a CSS-ből), a hangnemet (a szövegekből, AI segítségével), és a képi világ leírását (vision AI-val a meglévő képeken). Mindezt eltárolja az adatrétegben mint az új felhasználó brand kit-jét, 1. verzióként.
- Brand kit átnézés.** A felhasználó egy editor felületen megnézi, mit talált a rendszer. Színeket átszínezhethet, jobb logót feltölthet, a hangnem-leírást szerkesztheti. Mentésnél létrejön egy új verzió.
- Első brief.** A rendszer felkínál egy generikus briefet (pl. "Mutassuk meg, mi tesz minket különlegessé"), amit a felhasználó szerkeszthet vagy egyből elindíthat.
- Négy kreatív elkészül.** A háttér-feldolgozó elindít egy feladatot, az AI réteg kitalálja a négy variánst, a render réteg legenerálja a képeket, és a felhasználó látja őket egyenként megjelenni a felületen. Kb. 60 másodperc.

7. **Jóváhagyás vagy szerkesztés.** A felhasználó minden kreatívot jóváhagyhat, elvethet, vagy a szövegét inline átírhatja és újrarendereltetheti.

Folyamat 2 — Egy brief végigfut a rendszeren

Amikor egy meglévő felhasználó új briefet küld:

8. A frontend a briefet bejegyzi az adatrétegbe, és üzenetet küld a háttér-feldolgozónak: "új brief, dolgozd fel".
9. A háttér-feldolgozó elindítja a feladatot. Először betölti a brandhez tartozó aktuális brand kitet, és a brand utóbbi jóváhagyott kreatívjait (mint példák az AI-nek).
10. Meghívja az AI réteget az orchestrátor prompttal: "itt a brand kit, itt vannak a jó példák, itt a brief, adj nekem négy poszt-tervet". Az AI strukturált válaszként visszaad négy variánst: melyik sablont használja, mi legyen benne a szöveg, milyen kép kell hozzá, milyen színvariáció.
11. A háttér-feldolgozó párhuzamosan elindít négy alfeladatot. Mindegyik:
 - • legenerálja vagy lekeresi a képet (vagy a brand saját képtárából, vagy stock képből, vagy Flux-szal generálva)
 - • meghívja a render réteget, hogy állítsa össze a kész posztot a sablonból, a brand kitből és a kép + szöveg + színek inputból
 - • elmenti a kész képet a fájl tárolóba
 - • létrehoz egy kreatív rekordot az adatrétegben, amire a felhasználó hivatkozhat
12. A frontend valós időben látja, ahogy a kreatívok elkészülnek, és megjeleníti őket. A felhasználónak nem kell frissítenie az oldalt.

Folyamat 3 — Egy poszt kikerül Instagramra

Amikor a felhasználó egy jóváhagyott kreatívot kipoztol vagy beütemez:

13. A frontend bejegyzi az adatrétegbe egy "publikálási feladat" rekordot, megadott időponttal.
14. A háttér-feldolgozó percenként ellenőrzi: van-e olyan publikálási feladat, aminek itt az ideje. Ha igen, felveszi.
15. A háttér-feldolgozó meghívja a Meta API-t (Instagram és Facebook közös felülete). Először feltölti a képet és a caption (egy "container"-t hoz létre), majd egy második hívással publikálja.
16. Ha sikeres: bejegyzi az adatrétegbe a publikálás időpontját és az Instagram poszt azonosítóját. Ha sikertelen: rögzíti a hibát, és ha tranziens (pl. átmeneti hálózati hiba), automatikusan újrapróbálja.
17. A frontend mutatja a kreatív státuszát: "ütemezve / kipoztolva / hibára futott".

4. A komponensek részletesebben

A brand kit — a rendszer szíve

A brand kit nem egyetlen dolog, hanem egy strukturált csomag, ami mindent leír, ami a márka vizuális és nyelvi megjelenéséhez kell. Tartalma:

- Színek: elsődleges, másodlagos, kiemelő színek HEX kódokkal, és szabályok arra, mikor melyiket használjuk
- Tipográfia: betűtípus(ok), hierarchia (cím, alcím, szövegtörzs), maximális sorhossz

- Logó variánsok: világos / sötét háttérre, vízszintes / négyzet / ikon, biztonsági zónával
- Hangnem: leíró címkék (pl. "játékos", "direkt", "meleg") + példa posztok jó és rossz hangnemmél
- Képi szabályok: "mindig felülnézet, meleg tónusok, emberek nélkül" és hasonló, AI-promptként és negatív promptként is használható formában
- Referencia rács: 20–50 korábbi poszt, jó / rossz címkével — később ezzel finomhangolhatunk modellt egyedi márkára

A brand kitet verziózva tároljuk. Soha nem írunk felül egy korábbi verziót: ha a felhasználó módosít, új verzió jön létre. Így egy hat hónappal ezelőtti posztot ma is reprodukálni tudunk az akkori brand kittel.

A sablonok (templates)

A sablonok HTML és CSS fájlok. Ez kulcsfontosságú: nem egy AI generálja őket, hanem előre megtervezett, márka-független layoutok, amikbe behelyettesítjük a brand-specifikus elemeket (szín, font, logó) és a tartalmat (szöveg, kép).

Minden sablon:

- Pontos pixel méretű (pl. 1080×1350 az Instagram feed posztra)
- Tartalmaz "slot"-okat: ide jön a fő szöveg, ide a CTA, ide a háttérkép, ide a logó
- CSS változókat használ minden színhez és fonthoz — ezeket a brand kitből töltjük be
- Determinisztikus: ugyanaz az input mindig ugyanazt a kimenetet adja

Kezdetben kb. 20–30 sablon lesz a könyvtárban (quote, termék, bejelentés, testimonial, lista, story, carousel slide stb.). Az AI nem tervez új sablont — kiválasztja, melyik illik leginkább a briefhez, és kitölti.

Az AI orchestrátor

Ez a komponens hívja a nagy nyelvi modellt (Claude Sonnet) a generálási döntésekhez. Az input három dolog: a brief, az aktuális brand kit, és 5–10 korábbi jóváhagyott kreatív ugyanattól a brandtől (mint példa).

A kimenet strukturált JSON: négy variáns, mindegyikhez kiválasztott sablon, kitöltött szövegmezők, képgenerálási prompt (pozitív és negatív), színvariáció. Ezt a kimenetet a háttér-feldolgozó közvetlenül továbbadja a render rétegnek.

Fontos: az AI nem rajzol képet, nem helyezi el a logót, nem választ pontos hexa színt. Csak dönt: mi legyen rajta, milyen hangulatú, milyen sablon illik hozzá. A precíziót a render réteg adja hozzá.

A képgenerálás

Háromféle forrásból jöhet a poszt háttérképe:

- Stock kép: szemantikus keresés egy lokálisan indexelt Unsplash / Pexels gyűjteményen. Olcsó, gyors, sokszor a legjobb választás.
- Generált kép: Flux 1.1 Pro modell (fal.ai-on keresztül). Ha a brand stílusához nem találunk megfelelő stock-ot, vagy ha egyedi képet akarunk. A render réteg fogja rá tenni a szöveget — soha nem a Flux ír szöveget, mert nem megbízható.
- Brand saját képtára: a felhasználó által feltöltött saját képek.

Premium funkcióként később lesz egy negyedik opció: a brand saját referencia képein finomhangolt LoRA modell, ami a brand vizuális stílusát tanulja meg. Ez 20 referencia képből és néhány dollár költségből áll, és drasztikusan emeli a generált képek márkahűségét.

A render szolgáltatás

Ez egy önálló szolgáltatás, ami egyetlen feladatot lát el: kap egy bemenetet (melyik sablon, milyen szövegekkel és képpel, milyen brand kittel, milyen méretben), és visszaad egy URL-t a kész képhez.

Belül egy fejléc nélküli böngészőt használ (Playwright), ami megnyitja a sablon HTML-jét, beilleszti a CSS változókba a brand színeket és fontokat, a slot-okba a tartalmat, és aztán screenshot-ol pontos pixel méretben. A kész PNG felmegy a fájl tárolóba.

A render szolgáltatás cache-el: ha ugyanazt a bemenetet kapja kétszer, másodszor nem renderel újra, csak visszaadja a már meglévő URL-t. Ez sok pénzt és időt spórol.

A háttér-feldolgozó motor

Ez a komponens az események feldolgozója. Eseményeket fogad ("új brief", "publikálási feladat", "brand kinyerés"), és lépésekre bontva feldolgozza őket. Minden lépés:

- Tipizált és tesztelhető (TypeScript)
- Automatikusan retry-zol, ha tranzien hiba történik
- Idempotens: ha kétszer fut le, ugyanaz az eredmény
- Megfigyelhető: minden lépés állapota látható egy admin felületen

Ezt a komponenst azért nem n8n-ben építjük (annak ellenére, hogy a Think AI mindennap használja máshol), mert itt szigorú típus-, teszt- és verziókövetelmény van. Az n8n marad a Meta integráció oldali kerülő-utakra, a core pipeline ide kerül.

5. Az adatáramlás egy konkrét példán

Vegyünk egy konkrét helyzetet: Kovács Anna, aki egy budapesti kávézót üzemeltet, beír a rendszerbe, hogy "új tavaszi szezonális kávénk van, mutassuk meg". Lépésről lépésre:

18. Anna a frontend felületen beír egy briefet és nyom egy Generate gombot.
19. A frontend egy server-akcióval bejegyzí a briefet az adatrétegbe, és kiad egy *creative.requested* eseményt a háttér-feldolgozónak.
20. A háttér-feldolgozó felveszi az eseményt, és három dolgot tölt be: a brief részleteit, a kávézó aktuális brand kitjét (földszínek, kézzel írt fontok, meleg hangneme), és a kávézó utolsó 5 jóváhagyott posztját mint példát.
21. Meghívja Claude-ot egy strukturált prompittal. Claude visszaad négy variánst: egyik egy quote típusú "új szezonális ízek" felirattal, másik egy termékfókuszú slide, harmadik egy carousel-szerű meghívó, negyedik egy story formátum.
22. A háttér-feldolgozó négy párhuzamos feladatot indít. Mindegyik:
 - meghívja a fal.ai-t (vagy keres stock-ban), és kap egy háttérképet
 - meghívja a render szolgáltatást a sablonnal, a brand kittel, a szövegekkel és a háttérképpel
 - a render szolgáltatás visszaad egy PNG URL-t
 - bejegyez egy kreatív rekordot az adatrétegbe

23. Minden egyes kreatív elkészülésekor a frontend valós időben megjelenik a felületen (Supabase Realtime értesít).
24. Anna átnézi a négy posztot. Az elsőt egyből jóváhagyja, a másodikon átírja a szöveget ("szezonális ízek" → "új tavaszi kreációink") — a frontend újrahívja a render szolgáltatást, ami 2 másodperc alatt elkészíti a javított verziót.
25. Anna a jóváhagyott kreatívet beütemezi holnap reggel 9-re. A frontend létrehoz egy publikálási feladatot az adatrétegben.
26. Másnap reggel 9-kor a háttér-feldolgozó scheduled job-ja felveszi a feladatot, meghívja a Meta API-t, a poszt megjelenik az Instagramon. A háttér-feldolgozó bejegyzi az adatrétegbe a publikálás tényét és az Instagram poszt URL-jét.
27. Anna emailt kap (Resend-en keresztül), hogy a posztja kiment.

9. Mit nehéz jól csinálni, és mire figyelünk

Magyar ékezetek a betűtípusokban

Sok webfont nem tartalmazza az ő, ú, é, á, í karaktereket. Minden default sablonfontot auditálunk. A brand kit editorban van egy check: "őűéái" renderelése a választott fontban, és figyelmeztetés ha hiányzik glyph.

"AI-szlop" érzés

A generált képek könnyen szétesnek, ha hagyjuk a Flux-ot kötetlenül. A brand imagery rules-okat mindig betápláljuk a negative promptba ("emberek, arcok, plastik felszín, generikus iroda"). Visszacsatolás: az elvetett kreatívok promptjait elemezzük, a rossz patterneket hozzáadjuk a negative prompt templatehez.

Meta token lejárat

A long-lived Facebook token 60 napig él. Proaktívan frissítjük a 45. napon. Ha mégis lejár, egyértelmű "connect again" bannert mutatunk.

Carousel koherencia

Ha a négy slide-ot függetlenül generáljuk, vizuális káoszt kapunk. Az orchestrátor az egész carouselt egyetlen strukturált outputként tervezi: közös színkezelés, narratív ív a slide-okon át, az utolsó slide visszautal az elsőre.

Cold start a render szolgáltatáson

A Playwright konténer cold start 5–10 másodperc. Tartunk minimum 2 példányt mindig melegen, és terhelés alatt skálázunk fel. A felhasználó számára a render mindig pár másodperc, sose 10.

12. Fejlesztési alapelvek

Néhány elv, amit a kódbázisban és a fejlesztési kultúrában érvényesítünk:

- TypeScript strict mode. Nincs any típus, nincs ts-ignore. A típusok dokumentációként szolgálnak.
- Egy feladat — egy commit — egy pull request. Kisebb darabok könnyebben felülvizsgálhatók és könnyebben visszavonhatók.

- Server-first. A Next.js Server Component-jeit használjuk default-ként, csak akkor megyünk kliens-oldalra, ha tényleg kell.
- Zod validation minden API határon. Soha ne bízz a bemenetben, mindig validáld.
- Tesztek a kritikus folyamatokra. Nem 100% coverage a cél, hanem hogy a fontos user journey-k automatizáltan tesztelve legyenek.
- Tervezünk a hibára. Minden API hívás try/catch-csel. Minden külső szolgáltatás meghiúsodhat. Minden hosszú feladat újrapróbálható.
- Verziózunk mindent, ami a felhasználó számára visszanezhető (brand kit, kreatív, template).

belső dokumentáció